

Contents

01 - 入門	2
必要環境	3
建置步驟	3
專案初始化	3
第一個程式	3
註解的寫法	3
02 - 變數與型別	4
變數宣告	4
使用 @const 宣告不可變變數（常數）	4
型別標註	4
基本型別	5
字串操作	5
跳脫序列	5
重新賦值規則	6
元組解構	6
03 - 運算子	7
算術運算子	7
比較運算子	7
邏輯運算子	8
位元運算子	8
複合賦值運算子	9
遞增 / 遞減運算子	9
型別提升規則	9
歸屬運算子	10
控制流程	10
if / elif / else	10
while 迴圈	11
for 迴圈與 range	11
break 與 continue	12
巢狀範例	12
作用域規則	13
match	13
函式	14
基本函式定義	15
函式呼叫	15
遞迴函式	15
函式多載	15
省略回傳值型別（Unit 型別）	16
結構體與列舉型別	16
使用 record 定義結構體	16
建構式的使用方式	16
欄位存取（點記法）	16
欄位賦值	16
結構體作為函式參數	17
巢狀結構體	17
列舉型別（enum）	17
集合	18

元組	18
列表	19
映射	22
集合	23
進階功能	23
Lambda 函式	24
閉包	24
高階函式	24
將函式作為值使用	25
UFCS (Uniform Function Call Syntax)	25
運算子多載	25
Option 型別	26
F-String (字串插值)	26
型別轉換 (as)	27
帶關聯資料的 enum (ADT)	27
泛型 enum	28
Result 型別	28
套件	29
from/import 語法	29
子目錄 (點分隔)	29
目錄套件	29
標準函式庫 (std)	29
搜尋路徑的優先順序	30
PYTHONPATH 環境變數	30
限制事項	30
契約式設計	30
前置條件 (require)	30
後置條件 (ensure)	31
同時使用 require 和 ensure	31
結構體不變量 (invariant)	31
規則	32
測試	32
執行測試	32
撰寫測試	32
匹配器	33
輸出格式	33
模擬 (Mock)	33
參數化測試	34
基於屬性的測試	34
限制事項	34

[English](#) | [日本語](#) | [繁體中文](#)

01 - 入門

下一個教學 → [02 - 變數與型別](#)

必要環境

要建置並執行 Ry，您需要以下環境：

- LLVM 21
 - CMake 3.20 以上
 - 支援 C++17 的編譯器 (GCC 7+ / Clang 5+ 等)
-

建置步驟

在儲存庫根目錄下執行以下指令：

```
cmake -B build -DLLVM_DIR=/usr/local/llvm/lib/cmake/llvm
cmake --build build
```

建置成功後，會產生 build/ry 執行檔。

專案初始化

使用 ry init 指令建立新專案。

```
mkdir my-project
cd my-project
ry init
```

這將產生以下檔案和目錄：

- ry.toml — 專案設定檔
- src/main.ry — 進入點（附帶範例程式碼）
- test/ — 測試程式碼目錄

詳情請參閱[專案管理](#)。

第一個程式

將以下內容儲存為 hello.ry 檔案。

```
print("Hello, World!")
```

使用以下指令執行：

```
./build/ry hello.ry
```

輸出：

```
Hello, World!
```

註解的寫法

從 # 到行尾的內容會被視為註解。

```
# 這是一段註解
print("Hello") # 也可以在行尾加上註解
```

註解不會影響程式碼的執行。

下一個教學 → [02 - 變數與型別](#)

[English](#) | [日本語](#) | [繁體中文](#)

02 - 變數與型別

← [01 - 入門](#) / 下一個 → [03 - 運算子](#)

變數宣告

在 Ry 中，使用簡單的賦值語法宣告變數。預設情況下，變數是可變的。

```
x = 42          # 推論為 int 型別
y = 3.14        # 推論為 float 型別
flag = true     # 推論為 bool 型別
name = "Ry"     # 推論為 str 型別
```

使用 @const 宣告不可變變數（常數）

@const 指令將變數標記為不可變（常數）。宣告後無法變更其值。

```
@const
x = 42          # 推論為 int 型別

@const
y = 3.14        # 推論為 float 型別

@const
flag = true     # 推論為 bool 型別

@const
name = "Ry"     # 推論為 str 型別
```

型別標註

可以明確指定變數的型別。

```
@const
x: int = 42

@const
rate: float = 0.5

@const
ok: bool = false

@const
msg: str = "hello"
```

當型別標註與實際值的型別不一致時，會產生編譯錯誤。

基本型別

型別	說明	字面值範例
int	64 位元整數	0, 42, -10
byte	無號 8 位元整數 (0-255)	b: byte = 42
float	64 位元浮點數	0.0, 3.14, -1.5
bool	布林值	true, false
str	字串	"hello", ""

字串操作

字串支援多種操作。

```
@const
a = "Hello"
@const
b = "World"

# 串接
@const
c = a + ", " + b    # "Hello, World"

# 比較 (字典序)
print(a == b)    # false
print(a != b)    # true
print(a < b)     # true ("H" < "W")

# 長度
print(len(a))    # 5

# 子字串檢查
@const
s = "Hello, World!"
print(contains(s, "World"))    # true
print(starts_with(s, "Hello")) # true
print(ends_with(s, "!"))      # true
```

跳脫序列

字串中可使用以下跳脫序列。

序列	意義
\n	換行
\r	回車
\t	Tab
\\	反斜線
\"	雙引號
\0	空字元

```
print("Hello\nWorld")    # 分成兩行輸出
print("A\tB")            # 以 Tab 分隔
print("say \"hi\"")       # 包含雙引號的字串
```

重新賦值規則

未使用 `@const` 宣告的變數可以重新賦值，但有以下限制：

```
x = 10
x = 20          # OK: 重新賦予相同型別的值
# x = "text"    # 錯誤: 禁止變更型別的重新賦值
```

`@const` 無法重新賦值。

```
@const
N = 5
# N = 6    # 錯誤: 禁止對 @const 變數重新賦值
```

也無法重新宣告同名的變數。

```
x = 1
# 同一作用域內禁止重新宣告同名變數
```

元組解構

可以在單次宣告中將元組拆解為多個變數。

```
@const
a, b = (10, 20)
print(a)    # 10
print(b)    # 20
```

萬用字元

使用 `_` 忽略特定位置的值。

```
@const
x, _ = (1, 2)    # 只綁定 x; 2 被捨棄
print(x)         # 1
```

可變變數解構

省略 `@const` 即可宣告可變變數。

```
a, b = (10, 20)
a = 99
print(a)    # 99
```

規則

- 左側的變數數量必須與元組的元素數量相符。
 - 每個變數遵循與一般宣告相同的 `@const`/可變規則。
 - 不支援巢狀元組解構。
-

[← 01 - 入門](#) / [下一個](#) → [03 - 運算子](#)

[English](#) | [日本語](#) | [繁體中文](#)

03 - 運算子

[← 02 - 變數與型別](#) / [下一個](#) → [04 - 控制流程](#)

算術運算子

運算子	說明	範例	結果
+	加法	3 + 2	5
-	減法	3 - 2	1
*	乘法 / 字串重複	3 * 2	6
/	除法 (始終為 float)	7 / 2	3.5
//	整數除法 (始終為 int)	7 // 2	3
%	取餘	7 % 3	1
**	次方 (始終為 float)	2 ** 10	1024.0

```
@const
a = 10
@const
b = 3

print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.3333... (float)
print(a // b)   # 3 (int)
print(a % b)    # 1
print(2 ** 8)   # 256.0 (float)
```

比較運算子

比較運算子皆回傳 bool 值。

運算子	說明	範例
==	等於	a == b
!=	不等於	a != b
<	小於	a < b
<=	小於等於	a <= b
>	大於	a > b
>=	大於等於	a >= b

```
@const
x = 5
@const
y = 10
```

```

print(x == y)    # false
print(x != y)    # true
print(x < y)     # true
print(x <= y)    # true
print(x > y)     # false
print(x >= y)    # false

```

字串也可使用比較運算子（字典序比較）。

```

print("abc" == "abc")    # true
print("abc" < "abd")     # true
print("b" > "a")         # true

```

邏輯運算子

運算子	說明	範例
and	邏輯 AND	a and b
or	邏輯 OR	a or b
not	邏輯 NOT	not a

```

@const
t = true
@const
f = false

print(t and f)    # false
print(t or f)     # true
print(not t)      # false
print(not f)      # true

```

位元運算子

位元運算子僅適用於 int 型別。

運算子	說明	範例
&	位元 AND	5 & 3 → 1
\	位元 OR	5 \ 3 → 7
^	位元 XOR	5 ^ 3 → 6
~	位元 NOT (一元)	~5 → -6
<<	左移	1 << 3 → 8
>>	算術右移	8 >> 2 → 2
>>>	邏輯右移	-1 >>> 1 → 9223372036854775807

```

@const
a = 0b1010    # 10
@const
b = 0b1100    # 12

print(a & b)    # 8 (0b1000)
print(a \| b)   # 14 (0b1110)

```



```
print(a ^ b)      # 6 (0b0110)
print(~a)         # -11
print(1 << 4)     # 16
print(32 >> 2)    # 8
```

複合賦值運算子

更新變數值時可使用的簡寫語法。

運算子	說明	等同的表達式
+=	加法賦值	x = x + n
-=	減法賦值	x = x - n
*=	乘法賦值	x = x * n
/=	除法賦值	x = x / n
%=	取餘賦值	x = x % n

```
x = 10
x += 5      # x == 15
x -= 3      # x == 12
x *= 2      # x == 24
x /= 4      # x == 6.0 (會變為 float)
```

遞增／遞減運算子

將變數增減 1 的簡寫語法。

運算子	說明	等同的表達式
x++	加 1	x = x + 1
x--	減 1	x = x - 1

```
count = 0
count++      # count == 1
count++      # count == 2
count--      # count == 1
```

注意：僅可作為陳述式使用，不能在運算式中使用。

型別提升規則

以下說明運算中 int 與 float 混合時的行為。

```
# + - * 當其中一方為 float 時結果為 float
print(1 + 2)      # 3 (int)
print(1 + 2.0)    # 3.0 (float)
print(1.0 + 2)    # 3.0 (float)

# / 始終為 float
print(4 / 2)      # 2.0 (float)
```

```

# // 始終為 int
print(7 // 2)      # 3 (int)
print(7.0 // 2)    # 3 (int)

# ** 始終為 float
print(2 ** 3)      # 8.0 (float)

# % 當兩邊皆為 int 時為 int，其中一方為 float 時為 float
print(7 % 3)       # 1 (int)
print(7.5 % 2)     # 1.5 (float)

# + 當兩邊皆為 str 時為字串串接
print("foo" + "bar") # "foobar"

# * 當一方為 str、另一方為 int 時為字串重複
print("ab" * 3)     # "ababab"
print(3 * "ab")     # "ababab"

```

歸屬運算子

運算子	說明	範例
in	歸屬檢查	2 in {1, 2, 3} → true
not in	否定歸屬檢查	4 not in {1, 2, 3} → true

```

@const
s = {1, 2, 3}
print(2 in s)      # true
print(4 not in s)  # true

```

← [02 - 變數與型別](#) / 下一個 → [04 - 控制流程](#)

[English](#) | [日本語](#) | [繁體中文](#)

控制流程

← [前一篇：運算子](#) | [下一篇：函式](#) →

if / elif / else

使用 if 根據條件進行分支處理。

```

@const
x = 10

if x > 0:
    print(x)
elif x == 0:
    print(0)
else:
    print(-1)

```

- `elif` 和 `else` 可以省略。
- 條件式不限於 `bool`，也可以指定其他型別。對於 `int`，0 視為假，非 0 視為真。
- `if` 可以巢狀使用。

```
@const
a = 5
@const
b = 3

if a > 0:
    if b > 0:
        print(a + b)    # 8
```

while 迴圈

當條件為真時，重複執行區塊。

```
i = 3
while i > 0:
    print(i)
    i = i - 1
# 3
# 2
# 1
```

for 迴圈與 range

可使用列表或 `range` 進行迭代。

```
for x in [1, 2, 3]:
    print(x)
# 1
# 2
# 3
```

`range(n)` 產生從 0 到 `n - 1` 的整數。

```
for i in range(5):
    print(i)
# 0
# 1
# 2
# 3
# 4
```

`range(start, end)` 產生從 `start` 到 `end - 1` 的整數。

```
for i in range(2, 5):
    print(i)
# 2
# 3
# 4
```

`..` 範圍運算子建立包含兩端的範圍。1 .. 3 產生 [1, 2, 3]。

```
for i in 1 .. 3:
    print(i)
# 1
# 2
# 3
```

使用 `for k, v in map` 可以走訪映射的鍵值對。

```
@const
m = {"x": 10, "y": 20}
for k, v in m:
    print(k)
    print(v)
```

break 與 continue

`break` 立即跳出迴圈。`continue` 跳過目前的迭代，進入下一次迭代。

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
# 1
# 3
```

在 `while` 中也可同樣使用。

```
n = 0
while true:
    n = n + 1
    if n % 2 == 0:
        continue
    if n > 7:
        break
    print(n)
# 1
# 3
# 5
# 7
```

注意：在巢狀迴圈中，`break` / `continue` 僅作用於最內層的迴圈。在迴圈外使用會產生編譯錯誤。

巢狀範例

`for` 和 `while` 可以巢狀使用。

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i * 10 + j)
# 11
# 13
```

```
# 21
# 23
# 31
# 33
```

作用域規則

控制流程的區塊具有作用域。

區塊作用域

在區塊內宣告的變數無法從區塊外部參照。

```
if true:
    @const
    inner = 42
# 在此處參照 inner 會產生編譯錯誤
```

參照與重新賦值外部變數

可以從區塊內參照和重新賦值外部的變數。

```
count = 0
for i in range(5):
    count = count + i
print(count) # 10
```

內層作用域的重新賦值

在區塊內對變數賦值會修改外層的變數（Python 風格的作用域）。不會產生遮蔽——內層的賦值會修改同一個變數。

```
@const
x = 1
if true:
    x = 99
    print(x) # 99
print(x) # 99
```

match

match 是根據值進行分支的語法，可安全地處理 enum 和 Option。

```
enum Color:
    Red
    Green
    Blue

@const
c = Color::Green
match c:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
```

```

    case Color::Blue:
        print("blue")
# green

```

Option 的匹配

使用 `match` 可以安全地處理 `None` 的情況。

```

@const
x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")
# 42

```

萬用字元與字面值

`_` 是可匹配任何值的萬用字元模式。也可以使用字面值（數值、字串、布林值）進行匹配。

```

@const
n = 5
match n:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")
# other

```

guard 子句

可以使用 `if` 新增守衛條件。

```

match n:
    case x if x > 0:
        print("positive")
    case x if x < 0:
        print("negative")
    case _:
        print("zero")

```

注意：`match` 必須涵蓋所有模式。enum 必須包含所有變體，Option 必須包含 `Some` 和 `None`，字面值則需要 `_`。

[← 前一篇：運算子](#) | [下一篇：函式](#) →

[English](#) | [日本語](#) | [繁體中文](#)

函式

[← 前一篇：控制流程](#) | [下一篇：結構體](#) →

基本函式定義

函式使用 `fn` 關鍵字定義。參數的型別宣告為必要項，以 `name: type` 的格式指定。回傳值型別在 `->` 之後指定。

```
fn add(a: int, b: int) -> int:
    return a + b
```

- 參數的型別宣告是必要的。
 - 回傳值型別在 `->` 之後指定。
 - 使用 `return` 陳述式回傳值。
-

函式呼叫

透過名稱和參數來呼叫已定義的函式。

```
fn multiply(x: int, y: int) -> int:
    return x * y
```

```
@const
result = multiply(3, 4)
print(result)    # 12
```

遞迴函式

函式可以呼叫自身（遞迴）。

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

```
print(factorial(5))    # 120
print(factorial(0))    # 1
```

函式多載

可以定義多個同名但參數數量或型別不同的函式。

```
fn add(a: int, b: int) -> int:
    return a + b
```

```
fn add(a: float, b: float) -> float:
    return a + b
```

```
print(add(1, 2))        # 3
print(add(1.5, 2.5))    # 4
```

呼叫時會根據參數的型別自動選擇適當的函式。

注意：若定義參數型別相同但僅回傳值型別不同的函式，會產生編譯錯誤。

省略回傳值型別 (Unit 型別)

不需要回傳值的函式可以省略 `->`。此時函式回傳 Unit 型別。

```
fn greet():  
    print(42)  
  
greet()    # 42
```

這是最簡單的無參數、無回傳值函式形式。

[← 前一篇：控制流程](#) | [下一篇：結構體](#) →

[English](#) | [日本語](#) | [繁體中文](#)

結構體與列舉型別

[← 前一篇：函式](#) | [下一篇：集合](#) →

使用 record 定義結構體

使用 record 關鍵字定義結構體。各欄位以 `name: type` 的格式描述。

```
record Point:  
    x: int  
    y: int
```

結構體是堆疊上的值型別。

建構式的使用方式

像呼叫函式一樣使用結構體名稱來產生實例。參數按照欄位的定義順序指定。

```
@const  
p = Point(10, 20)
```

欄位存取 (點記法)

使用點記法存取欄位。

```
@const  
p = Point(10, 20)  
print(p.x)    # 10  
print(p.y)    # 20
```

注意：將結構體直接傳給 `print` 會產生錯誤。請個別傳遞欄位。

欄位賦值

未使用 `@const` 宣告的可變變數的欄位可以重新賦值。


```
p = Point(10, 20)
p.x = 100
print(p.x)    # 100
```

注意：對 `@const` 宣告的變數的欄位賦值會產生編譯錯誤。

結構體作為函式參數

可以將結構體作為函式的參數傳遞。

```
record Point:
  x: int
  y: int

fn distance_x(a: Point, b: Point) -> int:
  return a.x - b.x

@const
p1 = Point(10, 3)
@const
p2 = Point(4, 7)
print(distance_x(p1, p2))    # 6
```

巢狀結構體

結構體的欄位可以使用其他結構體。

```
record Point:
  x: int
  y: int

record Line:
  start: Point
  end: Point

@const
line = Line(Point(0, 0), Point(10, 5))
print(line.start.x)    # 0
print(line.end.x)      # 10
```

透過鏈結點記法可存取巢狀欄位。

列舉型別 (enum)

使用 `enum` 關鍵字定義列舉型別。每個變體作為具名常數處理。

定義

```
enum Color:
  Red
  Green
  Blue
```

使用方式

使用 `::` 存取變體。

```
@const
c = Color::Red
print(c)    # Red
```

比較

可以使用 `==` 和 `!=` 比較變體。

```
if c == Color::Red:
    print("red!")
elif c == Color::Green:
    print("green!")
else:
    print("blue!")
```

函式參數

可以使用 `enum` 名稱作為函式的參數型別。

```
fn describe(c: Color) -> str:
    if c == Color::Red:
        return "warm"
    return "cool"
```

[← 前一篇：函式](#) | [下一篇：集合](#) →

[English](#) | [日本語](#) | [繁體中文](#)

集合

[← 前一篇：結構體](#) | [下一篇：進階功能](#) →

Ry 有四種集合型別：元組、列表、映射、集合。

元組

元組是將多個值合為一體的不可變資料結構，可以持有不同型別的元素。

建立

```
@const
t = (1, 3.14)
```

型別標註

```
@const
t: (int, float) = (1, 3.14)
```

元素存取

使用 `.0`、`.1`、`...` 等索引來存取。

```
@const
t = (1, 3.14)
print(t.0)    # 1
print(t.1)    # 3.14
```

函式回傳值

想要回傳多個值時，元組很方便。

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
@const
result = swap(1, 2)
print(result.0) # 2
print(result.1) # 1
```

限制事項

- 超出範圍的索引（例如：對只有 2 個元素的元組使用 `.2` 存取）會產生編譯錯誤。
 - 將元組直接傳給 `print` 會產生錯誤。請個別傳遞各元素。
-

列表

列表是由相同型別的元素排列而成的可變長度資料結構。

建立

```
@const
xs = [1, 2, 3]
```

型別標註

```
@const
xs: List<int> = [1, 2, 3]
```

索引存取

```
print(xs[0])    # 1
```

```
@const
i = 1
print(xs[i])    # 2
```

索引賦值

```
xs[0] = 99
```

len

```
print(len(xs))  # 3
```

print

```
print(xs)    # [1, 2, 3]
```

for 走訪

```
for x in xs:
    print(x)
```

函式參數

```
fn first(xs: List<int>) -> int:
    return xs[0]
```

filter、map、sort

串列支援 filter、map、sort 操作。這些操作會傳回新串列，不會修改原始串列。

```
@const
xs = [1, 2, 3, 4, 5]

# filter: 保留符合條件的元素
@const
evens = xs.filter(fn(x: int): x > 3)
print(evens)    # [4, 5]

# map: 轉換每個元素
@const
doubled = xs.map(fn(x: int): x * 2)
print(doubled)   # [2, 4, 6, 8, 10]

# sort: 升序排序（預設）
@const
sorted = [3, 1, 2].sort()
print(sorted)    # [1, 2, 3]

# 鏈接
@const
result = xs.filter(fn(x: int): x > 1).map(fn(x: int): x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce、fold

reduce 從第一個元素開始將串列歸約為單一值。fold 則可提供明確的初始值。

```
@const
xs = [1, 2, 3, 4, 5]

# reduce: 從第一個元素開始
@const
total = reduce(xs, fn(a: int, b: int): a + b)
print(total)    # 15

# fold: 提供明確的初始值
@const
total2 = fold(xs, 0, fn(a: int, b: int): a + b)
print(total2)    # 15
```

any、all

any 如果至少有一個元素滿足述詞則傳回 true。all 如果所有元素都滿足則傳回 true。

```
@const
xs = [1, 2, 3, 4, 5]

print(any(xs, fn(x: int): x > 4))    # true
print(any(xs, fn(x: int): x > 9))    # false

print(all(xs, fn(x: int): x > 0))    # true
print(all(xs, fn(x: int): x > 3))    # false
```

sum、min、max

```
@const
xs = [3, 1, 4, 1, 5]
print(sum(xs))    # 14
print(min(xs))    # 1
print(max(xs))    # 5
```

first、last、is_empty

```
@const
xs = [10, 20, 30]
print(first(xs))    # 10
print(last(xs))     # 30
print(is_empty(xs)) # false
```

enumerate、zip

enumerate 為每個元素附上索引。zip 將兩個串列逐元素合併。

```
@const
xs = [10, 20, 30]
@const
indexed = enumerate(xs)
# [(0, 10), (1, 20), (2, 30)]
for p in indexed:
    print(p.0)
    print(p.1)

@const
ys = ["a", "b", "c"]
@const
zipped = zip(xs, ys)
# [(10, "a"), (20, "b"), (30, "c")]
```

限制事項

- 所有元素必須是相同型別，不能混合不同型別。
- 空列表 [] 會產生錯誤。
- 超出範圍的存取會產生執行期錯誤 (exit(1))。
- 元素型別支援 int、float、bool、str。

映射

映射是管理鍵值對的關聯陣列。

建立

```
@const
m = {"a": 1, "b": 2}
```

型別標註

```
@const
m: Map<str, int> = {"a": 1, "b": 2}
```

鍵存取

```
print(m["a"])    # 1
```

插入 / 更新

對新的鍵賦值即為插入，對既有的鍵賦值即為更新。

```
m["c"] = 3      # 新增
m["a"] = 99     # 更新
```

len

```
print(len(m))   # 3
```

print

```
print(m)        # {a: 99, b: 2, c: 3}
```

has_key

確認鍵是否存在。

```
print(m.has_key("a"))  # true
```

keys、values

keys 傳回所有鍵的串列。values 傳回所有值的串列。

```
@const
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
print(values(m))  # [1, 2, 3]
```

函式參數

```
fn get_val(m: Map<str, int>, k: str) -> int:
    return m[k]
```

限制事項

- 所有鍵必須是相同型別，所有值也必須是相同型別。
- 空映射需要型別標註。
- 存取不存在的鍵會產生執行期錯誤 (exit(1))。

集合

集合是持有相同型別元素且不重複的集合型別。

建立

```
@const  
s = {1, 2, 3}
```

型別標註

```
@const  
s: Set<int> = {1, 2, 3}
```

in 運算子

使用 in 運算子確認元素是否包含在集合中。

```
print(2 in s)    # true  
print(5 in s)    # false
```

add / remove

```
s.add(4)          # 新增元素  
s.remove(1)       # 移除元素  
s.add(2)          # 已存在，因此忽略
```

len / print

```
print(len(s))    # 3  
print(s)         # {2, 3, 4}
```

for 走訪

```
for x in s:  
    print(x)
```

空集合

空集合需要型別標註。

```
@const  
empty: Set<int> = {}
```

限制事項

- 所有元素必須是相同型別。
- 元素型別支援 int、float、bool、str。

[← 前一篇：結構體](#) | [下一篇：進階功能](#) →

[English](#) | [日本語](#) | [繁體中文](#)

進階功能

[← 前一篇：集合](#) | [下一篇：套件](#) →

Lambda 函式

Lambda 函式是將函式以表達式形式撰寫的語法，以 `fn(參數): 表達式` 的形式書寫。回傳值型別會自動推論。

單一表達式 Lambda

```
@const
double = fn(x: int): x * 2
print(double(5)) # 10

@const
add = fn(a: int, b: int): a + b
print(add(3, 4)) # 7
```

無參數 Lambda

```
@const
answer = fn(): 42
print(answer()) # 42
```

多行 Lambda

在 `:` 後換行並縮排，即可撰寫多個陳述式。

```
@const
abs = fn(x: int):
    if x < 0:
        return -x
    return x

print(abs(-5)) # 5
print(abs(3)) # 3
```

閉包

Lambda 函式可以捕獲定義時作用域中的變數。

```
@const
offset = 10
@const
add_offset = fn(x: int): x + offset
print(add_offset(5)) # 15
```

高階函式

可以定義接受函式作為參數的函式。函式型別以 `fn(參數型別) -> 回傳值型別` 的形式書寫。

```
fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

@const
double = fn(x: int): x * 2
print(apply(double, 3)) # 6
print(apply(fn(n: int): n + 1, 10)) # 11
```

將函式作為值使用

具名函式也可以繫結到變數或作為參數傳遞。

```
fn square(x: int) -> int:
    return x * x

fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

# 將具名函式作為參數傳遞
print(apply(square, 4)) # 16

# 繫結到變數
@const
sq = square
print(sq(5)) # 25
```

UFCS (Uniform Function Call Syntax)

使用 UFCS 可以將 $f(a, b)$ 的呼叫寫成 $a.f(b)$ ，實現類似方法鏈的寫法。

```
fn add(a: int, b: int) -> int:
    return a + b

@const
x = 1
print(x.add(2)) # add(x, 2) → 3

鏈式呼叫

fn double(n: int) -> int:
    return n * 2

print(x.add(2).double()) # double(add(x, 2)) → 6
```

運算子多載

使用 `fn operator` 運算子 語法可為自訂型別定義運算子。

二元運算子

接受 2 個參數。

```
record Vec2:
    x: int
    y: int

fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

fn operator==(a: Vec2, b: Vec2) -> bool:
```

```

    return a.x == b.x and a.y == b.y

@const
v1 = Vec2(1, 2)
@const
v2 = Vec2(3, 4)
@const
v3 = v1 + v2
print(v3.x)      # 4
print(v1 == v2)  # false

```

一元運算子

接受 1 個參數。

```

fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)

```

支援的運算子一覽

類別	運算子
算術	+, -, *, /, %, **, //
比較	==, !=, <, <=, >, >=
位元	&, \, ^, ~, <<, >>
邏輯	and, or, not

Option 型別

表示值是否存在的型別，可以是 Some(值) 或 None。

```

@const
x: Option<int> = Some(42)
print(x)      # Some(42)

@const
y: Option<int> = None
print(y)      # None

```

取出值

使用 match 安全地取出內部的值，並處理 None 的情況。

```

match x:
    case Some(v):
        print(v)      # 42
    case None:
        print("nothing")

```

F-String (字串插值)

使用 f"..." 可以在字串中直接嵌入表達式。表達式放在 {} 內。

```

@const
name = "Alice"
print(f"Hello {name}")    # Hello Alice

@const
x = 3
@const
y = 4
print(f"{x} + {y} = {x + y}")    # 3 + 4 = 7

```

使用 `{` 和 `}` 來包含字面大括號。

```

print(f"{{escaped}}")    # {escaped}

```

型別轉換 (as)

使用 `as` 在型別之間進行明確轉換。

```

@const
x = 42 as float    # 42.0
@const
y = 3.14 as int    # 3 (截斷)
@const
s = 42 as str      # "42"
@const
b = true as int    # 1

```

帶關聯資料的 enum (ADT)

enum 變體可以攜帶關聯值，讓單一 enum 代表一系列不同形狀的資料。

```

enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

```

建構 ADT 變體

```

@const
c = Shape::Circle(3.14)
@const
r = Shape::Rectangle(4.0, 5.0)
@const
p = Shape::Point

```

匹配 ADT 變體

在 `case` 中使用綁定模式來取出關聯資料。

```

fn describe(s: Shape) -> str:
    match s:
        case Shape::Circle(r):
            return f"circle with radius {r}"
        case Shape::Rectangle(w, h):

```

```

        return f"rectangle {w}x{h}"
    case Shape::Point:
        return "point"

print(describe(Shape::Circle(3.14)))      # circle with radius 3.14
print(describe(Shape::Rectangle(4.0, 5.0))) # rectangle 4.0x5.0

```

泛型 enum

enum 可以帶有序別參數，使其可在不同載體型別間重複使用。

```

enum MyOption<T>:
    MySome(T)
    MyNone

```

使用方式

```

@const
a = MyOption<int>::MySome(42)
@const
b: MyOption<int> = MyOption<int>::MyNone

match a:
    case MyOption::MySome(v):
        print(v)      # 42
    case MyOption::MyNone:
        print("none")

```

Result 型別

Result<T, E> 用於可能失敗的函式。成功時回傳 Ok(value)，失敗時回傳 Err(error)。

```

fn divide(a: int, b: int) -> Result<int, str>:
    if b == 0:
        return Err("division by zero")
    return Ok(a // b)

```

使用 match 來處理結果。

```

@const
r = divide(10, 0)
match r:
    case Ok(v):
        print(v)
    case Err(e):
        print(e)      # division by zero

```

[← 前一篇：集合](#) | [下一篇：套件](#) →

[English](#) | [日本語](#) | [繁體中文](#)

套件

[← 前一篇：進階功能](#) | [下一篇：契約式設計](#) →

Ry 使用套件系統來管理跨檔案和目錄的程式碼。詳細規格請參閱[套件參考手冊](#)。

from/import 語法

使用 from 語法匯入其他檔案的函式。

```
from math import add, sub    # 選擇性匯入
from math                    # 全部匯入
```

這樣就可以使用 math.ry 中定義的函式。

子目錄（點分隔）

可以使用點分隔來指定子目錄中的套件。

```
from utils.math import add    # 匯入 utils/math.ry
```

每個點對應一層目錄分隔。

目錄套件

套件可以是單一的 .ry 檔案，也可以是包含多個 .ry 檔案的目錄。當套件解析為目錄時，其中所有的 .ry 檔案會自動載入。

```
mypackage/
  math.ry      # fn add(), fn sub()
  string.ry    # fn concat()

from mypackage          # 匯入 add、sub、concat
from mypackage import add # 僅匯入 add
```

不需要特殊的入口檔案（如 __init__.py）。以 _ 開頭的檔案會被排除。

標準函式庫（std）

std 套件會自動匯入到所有程式中。不需要撰寫 from std。

```
# 這些函式無需匯入即可使用
print("hello")
@const
n = len("world")
@const
xs = range(5)
```

也可以從 std 子套件中明確匯入特定定義：

```
from std.str import contains
```

RY_HOME

標準函式庫安裝在 `$RY_HOME/lib/std/`。RY_HOME 的預設值為 `~/ .ry`。

```
export RY_HOME="$HOME/.ry" # 預設
```

搜尋路徑的優先順序

套件檔案按以下順序搜尋：

1. 匯入來源檔案的目錄 — 首先搜尋撰寫匯入的檔案所在的目錄。
 2. `$RY_HOME/lib` — 標準函式庫的位置。
 3. 執行檔相對的 `lib/` — 相對於 `ry` 執行檔的目錄。
 4. `RY_PATH` 環境變數 — 找不到時，按順序搜尋 `RY_PATH` 中指定的目錄。
-

RY_PATH 環境變數

可以使用冒號分隔指定多個目錄。

```
export RY_PATH=/home/user/ry-libs:/usr/local/ry-libs
```

設定後，可以從任何地方匯入指定目錄中的套件。

限制事項

- `from` 陳述式只能寫在檔案的最上層，不能寫在函式或區塊內部。
- 多次匯入同一套件時，會自動跳過（不會發生重複匯入）。
- 循環匯入（A 匯入 B，B 匯入 A）會產生錯誤。

```
# 錯誤範例：a.ry 和 b.ry 互相匯入的情況
# a.ry: from b import foo
# b.ry: from a import bar ← 循環匯入錯誤
```

[← 前一篇：進階功能](#) | [下一篇：契約式設計](#) →

[English](#) | [日本語](#) | [繁體中文](#)

契約式設計

[← 前一篇：套件](#) | [下一篇：測試](#) →

Ry 支援 Eiffel 風格的契約式設計，包含前置條件 (`require`)、後置條件 (`ensure`) 以及結構體不變量 (`invariant`)。當契約違反時，程式會終止。詳細規格請參閱[契約式設計參考手冊](#)。

前置條件 (`require`)

使用 `require` 指定函式被呼叫時必須為真的條件。

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
```

```
    new_balance: int = balance + amount
    return new_balance
```

當前置條件不滿足時，程式會以下列訊息終止：

```
Contract violation: require failed in deposit()
```

後置條件 (ensure)

使用 `ensure` 指定函式回傳值時必須為真的條件。

result 關鍵字

在 `ensure` 區塊內，`result` 代表回傳值。

```
fn abs(x: int) -> int:
    ensure:
        result >= 0
    if x < 0:
        return -x
    return x
```

old() 表達式

`old(expr)` 捕獲函式開始時表達式的值。這對於比較狀態變化前後非常有用。

```
fn increment(x: int) -> int:
    ensure:
        result == old(x) + 1
    return x + 1
```

同時使用 `require` 和 `ensure`

兩者可以同時使用。`require` 必須寫在 `ensure` 之前。

```
fn deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    ensure:
        result >= 0
        result == old(balance) + amount
    new_balance: int = balance + amount
    return new_balance
```

結構體不變量 (invariant)

使用 `invariant` 指定結構體必須始終成立的條件。不變量會在建構後及每次欄位賦值後檢查。

```
record BankAccount:
    balance: int
    min_balance: int
    invariant:
        balance >= min_balance
```

```
@const
a = BankAccount(100, 0)    # OK: 100 >= 0
# a.balance = -1           # Contract violation: invariant failed
```

規則

- `require` 和 `ensure` 區塊為選用項，寫在函式主體之前。
 - 同時使用時，`require` 必須寫在 `ensure` 之前。
 - `result` 和 `old()` 只能在 `ensure` 區塊內使用。
 - `invariant` 寫在 `record` 定義的末尾，在所有欄位宣告之後。
 - 所有契約違反都會以 `exit(1)` 終止程式。
-

[← 前一篇：套件](#) | [下一篇：測試](#) →

[English](#) | [日本語](#) | [繁體中文](#)

測試

[← 前一篇：契約式設計](#)

Ry 內建了使用 `describe`、`it`、`expect` 的 RSpec 風格測試語法。詳細規格請參閱[測試參考手冊](#)。

執行測試

```
ry test                # 自動探索並執行所有 *.test.ry 檔案
ry test tests/spec     # 遞迴執行指定目錄下所有 *.test.ry 檔案
ry test tests/my_test.test.ry # 執行特定的測試檔案
```

所有測試通過時，結束碼為 0；若有任一測試失敗，則為 1。

不帶參數執行時，`ry test` 會搜尋 `ry.toml` 來找到專案根目錄，然後遞迴地探索所有 `*.test.ry` 檔案。

撰寫測試

使用 `describe` 將相關測試分組，使用 `it` 定義各個測試案例。

```
describe("Calculator", fn():
  it("adds integers", fn():
    expect(1 + 2).to_eq(3)
  )
  it("subtracts integers", fn():
    expect(5 - 3).to_eq(2)
  )
  it("checks booleans", fn():
    expect(3 > 1).to_be_true()
  )
)
```

- `describe` 和 `it` 接受描述字串和 `lambda` 引數 `fn()`：作為第二個參數。

- `describe / it / expect / mock / verify` 僅可在 `ry test` 中使用（一般的 `ry` 執行會產生編譯錯誤）。

匹配器

匹配器	說明	支援型別
<code>to_eq(expected)</code>	等值比較	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_not_eq(expected)</code>	不等值斷言	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_be_true()</code>	<code>true</code> 斷言	<code>bool</code>
<code>to_be_false()</code>	<code>false</code> 斷言	<code>bool</code>
<code>to_be_none()</code>	<code>None</code> 斷言	<code>Option</code>
<code>to_be_some()</code>	<code>Option</code> 為 <code>Some</code> 的斷言	<code>Option</code>
<code>to_contain(val)</code>	容器包含值的斷言	<code>List</code> , <code>Set</code> , <code>str</code>

輸出格式

Calculator

```
+ adds integers
+ subtracts integers
- checks booleans
line 10: expected true, got false
```

2 passed, 1 failed

+ 表示通過（綠色），- 表示失敗（紅色）。

模擬 (Mock)

`mock(fn_name, replacement)`

在目前的 `it` 區塊中將函式替換為模擬實作。 `it` 區塊結束時會自動還原。

```
fn fetch_data() -> str:
    return "real data"

describe("mocking", fn():
    it("replaces function", fn():
        mock(fetch_data, fn(): "fake")
        expect(fetch_data()).to_eq("fake")

    )
    it("auto-restores", fn():
        expect(fetch_data()).to_eq("real data")
    )
)
```

`verify(fn_name)`

傳回模擬函式被呼叫的次數。

```
describe("verify", fn():
  it("counts calls", fn():
    mock(fetch_data, fn(): "fake")
    fetch_data()
    fetch_data()
    expect(verify(fetch_data)).to_eq(2)
  )
)
```

參數化測試

使用 `@each` 以多組輸入執行同一個測試:

```
@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):
  expect(a + b).to_eq(expected)
)
```

每個元組成為一個獨立的測試案例。描述中的 `{0}`, `{1}` 會被替換為實際值。

基於屬性的測試

使用 `@property` 以隨機生成的輸入進行測試:

```
@property(count=100)
it("addition is commutative", fn(a: int, b: int):
  expect(a + b).to_eq(b + a)
)
```

測試以隨機值執行 `count` 次。失敗時會顯示反例。

限制事項

- 不支援 `describe` 的巢狀使用
 - 不支援 `before_each` / `after_each`
 - 多載函式及 `@native` 函式無法模擬
-

[← 前一篇：契約式設計](#)